Devoid Viewer

Codebase Documentation & Design Guide

This document explains every file, struct, method, and design decision in the Devoid Viewer codebase. It is written for someone new to Rust who wants to understand not only *what* the code does but *why* it was written that way — including the threading model, the choice of external libraries (crates), and the subtleties that come with building a responsive GUI application.

Table of Contents

Section	Topic	Page
1	Project structure overview	2
2	Cargo.toml — dependencies explained	2
3	nav.rs — directory navigation	5
4	loader.rs — image decoding	7
5	main.rs — application core	9
6	Threading & asynchronous design	17
7	Key design decisions & trade-offs	19
8	Caveats & known limitations	21
9	Glossary of Rust concepts used	22

1. Project Structure Overview

The project follows the standard Cargo layout. All hand-written source files live under **src/**. There are three modules, each with a single responsibility:

File	Responsibility
`Cargo.toml`	Declares the project name, version, and all external dependencies (crates).
`build.rs`	Build script — embeds a Windows application icon and metadata via the winres crate.
`src/main.rs`	Application entry point, winit event loop, app state, input handling, pixel rendering.
`src/loader.rs`	Image file decoding for every supported format. Runs on background threads.
`src/nav.rs`	Tracks which images exist in a directory and manages next/prev navigation.

2. Cargo.toml — Dependencies Explained

In Rust, external libraries are called **crates**. You declare them in **Cargo.toml** and Cargo downloads and compiles them automatically. Each crate was chosen deliberately for leanness — the final dependency graph is roughly 50–70 crates and the release binary around 2–3 MB, compared to 200+ crates and ~9 MB with a full GUI toolkit.

Windowing: winit

winit is the de-facto standard Rust crate for creating OS windows and receiving input events. It handles the event loop, keyboard and mouse input, window resize, drag-and-drop, and monitor queries. It has no rendering opinion — it simply gives you a window handle and events. This makes it extremely lean compared to a full GUI toolkit like egui.

The rwh_05 feature enables the raw-window-handle v0.5 interface that softbuffer requires to obtain the platform-specific surface pointer.

Pixel display: softbuffer

softbuffer gives you a Vec<u32> that maps directly to the window's pixels. You write colours into it and call present() — the OS compositor blits it to the screen. There is no GPU pipeline, no shader compilation, no texture atlas. For an image viewer that displays one static image at a time, this is both simpler and more memory-efficient than an OpenGL or Vulkan backend.

> ■ *softbuffer uses the platform's most efficient path internally — on Windows this is a DIB section (GDI), which is hardware-accelerated at the compositor level. You do not pay for a GPU context you don't need.*

Native dialogs: windows-sys

windows-sys is a thin, zero-overhead binding to the Windows API. We enable only two feature flags — Win32_UI_WindowsAndMessaging and Win32_Foundation — which give us MessageBoxW. This replaces any rendered modal dialog with a native OS dialog: no widget rendering code, no font renderer, and the result looks exactly as Windows users expect. It is a target-specific dependency

([target.'cfg(windows)'.dependencies]) so it does not affect non-Windows builds.

Image decoding: image

The **image** crate is the standard Rust library for raster image I/O. It supports BMP, PNG, JPEG, GIF, WebP, TIFF, and ICO. Features are enabled selectively to avoid compiling unused decoders. The crate decodes any supported format into a normalised RGBA8 byte buffer.

JPEG XL: jxl-oxide

JPEG XL (.jxl) is a next-generation format not yet supported by the image crate. **jxl-oxide** is a pure-Rust JXL decoder. We use the low-level streaming API (render.stream().write_to_buffer()) because the higher-level image feature had a private-method issue in version 0.10. The image feature flag is therefore not enabled, keeping the dependency graph slightly leaner.

SVG: resvg + usvg + tiny-skia

SVG is a vector format — it has no pixel data, only drawing instructions. To display it we must rasterise it into a pixel grid.

- **usvg** parses the SVG XML and normalises the scene tree.
- **resvg** walks the scene tree and renders each shape.
- **tiny-skia** is the 2D rasteriser that produces a premultiplied RGBA pixel buffer, which we unpremultiply before storing.

These three crates are the largest contributor to binary size after the image decoders, but they are the only correct way to handle SVG in Rust.

Channel communication: crossbeam-channel

crossbeam-channel provides faster, more ergonomic channels than std::sync::mpsc. We use a bounded channel of capacity 32 so that if the UI thread falls behind, background threads are throttled rather than consuming unbounded memory.

Windows resource embedding: winres (build dependency)

winres runs at build time only ([build-dependencies]). It embeds a .ico file and version metadata into the .exe using the Windows resource compiler. Zero runtime cost — purely a build-time tool.

3. nav.rs — Directory Navigation

This module is unchanged from earlier versions. It answers one question: given a file the user is currently looking at, what are the next and previous images in the same folder?

struct DirNavigator

- **files: Vec<PathBuf>** — a sorted list of every supported image file in the current directory.
- **index: usize** — the position of the currently viewed file.

Derives **Default** — Rust auto-generates an initialiser that sets files to an empty Vec and index to 0, removing the need for a manual constructor.

fn set_current(path: PathBuf)

Called every time a new image is opened. Checks whether the directory changed; if so, rebuilds the sibling list. Locates the exact path in the list and updates the index. Falls back to a single-entry list if the path is not found (e.g. a different drive with no other images).

fn rebuild(dir)

Reads the directory, filters for supported files, and sorts them using the custom **human_sort** comparator so that img2 sorts before img10 rather than the lexicographic img10 < img2.

fn next() / fn prev()

Advance or retreat with wrap-around. Both return Option<PathBuf>. prev() uses checked_sub(1) to safely handle wrap-around on an unsigned integer without risking overflow.

fn peek_next() / fn peek_prev()

Read-only lookahead used by **preload_adjacent()** to decode neighbouring images in the background before the user navigates to them.

fn remove(path)

Removes a path after deletion. Uses saturating_sub to adjust the index safely when the list becomes empty.

fn human_sort / fn consume_number

Character-by-character comparator. When both iterators point at digit characters, consume_number() reads the entire digit run as a u64 and compares numerically, ensuring img2 < img10.

> ■ *Rust iterators are lazy. Peekable<Chars> lets us inspect the next character without consuming it — essential for this algorithm.*

4. loader.rs — Image Decoding

This module decodes image files into raw RGBA8 bytes. It has no dependency on any GUI toolkit — it returns its own **RawImage** struct, which main.rs converts into a **DecodedImage** via the standard Rust From trait.

struct RawImage

A plain data struct holding the decode result:

- **pixels: Vec<u8>** — straight RGBA8 bytes, row-major. Every four bytes are one pixel: R, G, B, A.
- **width / height: u32** — pixel dimensions.

'Straight' (unassociated) alpha means RGB channels are independent of the alpha value. This is what the renderer's compositing formula expects.

type LoadResult

A type alias for Result<RawImage, Box<dyn Error + Send + Sync>>. The boxed trait object allows any error type to be returned without a shared error enum. *Send* + *Sync* bounds are required because errors may be created on background threads and sent to the UI thread.

fn is_supported(path) → bool

Fast extension check. Uses is_some_and() — returns false if the extension is absent, evaluates the membership test if it is present.

fn load_image(path) → LoadResult

Dispatches to the correct decoder by lowercased file extension. The catch-all _ arm sends everything unrecognised to *load_via_image_crate*.

fn load_via_image_crate — BMP, PNG, JPEG, GIF, WebP, TIFF, ICO

image::open() auto-detects format and decodes. into_rgba8() normalises to 8-bit RGBA regardless of source depth or colour model. The **raw_from_dynamic** helper extracts the raw bytes and wraps them in a RawImage.

> ■ *The ? operator propagates errors automatically: if image::open() fails, the function immediately returns that error to the caller.*

fn load_jxl — JPEG XL

- File opened in a BufReader for efficient buffered I/O.
- request_color_encoding(srgb(...)) forces sRGB output regardless of the embedded colour profile.
- render_frame(0) decodes the first animation frame.
- Stream yields interleaved f32 samples. The match block expands 1 (greyscale), 2 (grey+alpha), 3 (RGB), or 4 (RGBA) channels to full RGBA u8.
- **f32_to_u8** clamps to [0.0, 1.0] before scaling — prevents overflow on HDR content.

fn load_svg — SVG

- **usvg** parses and normalises the SVG scene tree.
- **resvg** renders the tree into a **tiny-skia** Pixmap.
- tiny-skia uses premultiplied alpha. We divide each channel by alpha to recover straight alpha. Pixels with alpha == 0 are zeroed to avoid division by zero.

5. main.rs — Application Core

This file contains the winit event loop, all application state, and the pixel renderer. There is no widget toolkit, no font renderer, and no GPU pipeline. All design decisions in this version prioritise explicitness and minimal state — every field, every return value, every type carries a clear, single purpose.

5.1 const BG: u8 = 42

The background grey level used in two places: filling the blank state and compositing transparent image regions. Naming it avoids the magic number 42 appearing multiple times in the render loop, and makes it trivial to change the background colour in one place.

5.2 struct DecodedImage

The in-memory representation of a displayable image — straight RGBA8 pixels plus dimensions. Derives both **Clone** and **Debug**. Clone is needed because apply_image stores the image as current_image and simultaneously needs to read its dimensions to return them. Debug is required because LoadMsg derives Debug, and all types it contains must also implement Debug.

impl From<loader::RawImage> for DecodedImage — the standard Rust conversion pattern. This means the background thread can write raw.into() after decoding, and Rust automatically calls this implementation. No bespoke constructor or explicit field copying needed at the call site.

5.3 enum OpenMode

Two variants: **FirstImage** (resize the window to the image) and **Navigate** (keep the existing window size). Using an enum instead of a bool makes every call site self-documenting — OpenMode::FirstImage is unambiguous; true would require the reader to look up the parameter name.

5.4 enum LoadMsg

Three clean variants, each with a single purpose:

- **Loaded** — primary decode complete. Carries the path, a DecodedImage, and a resize: bool derived from OpenMode at the time the load was initiated.
- **Preloaded** — speculative decode complete. Carries the path and image; goes directly into the cache.
- **Error** — decode failed. Carries the path and an error string.

All types inside LoadMsg implement Send automatically (Vec<u8>, u32, PathBuf, bool, String are all Send by default). No unsafe impl Send is needed or appropriate here — writing one would bypass the compiler's automatic safety analysis without providing any benefit.

5.5 struct WakeUp / type Proxy

WakeUp is a zero-size marker struct used as the custom user-event type for the EventLoop<WakeUp>. Background threads call proxy.send_event(WakeUp) to inject a synthetic event and wake the sleeping event loop after posting a LoadMsg. Without this, the loop would sleep indefinitely waiting for OS input.

Proxy is a type alias for winit::event_loop::EventLoopProxy<WakeUp>. The alias avoids repeating this verbose type throughout the code.

5.6 struct App — fields

- **current_path / current_image** — what is on screen. current_image is deliberately NOT cleared when navigation starts — the previous image stays visible until the new one is decoded, giving seamless browsing with no blank flashes.
- **loading_path** — the path currently being decoded. Used to discard stale channel messages from loads we navigated away from.
- **preload_cache** — decoded neighbours ready for instant display. CPU memory only — no GPU upload step exists in this architecture.
- **pending_center** — triggers window centering on the event after a resize command has been issued (see Section 6.5).

Notably absent: error_msg, show_delete_dialog, and needs_redraw — all three existed in earlier versions but were removed. Error messages are shown immediately via show_error_dialog(). The delete dialog is a blocking MessageBoxW call that freezes the event loop, so no flag is needed to suppress navigation. And winit only fires RedrawRequested when request_redraw() was explicitly called, so a guard flag is redundant.

5.7 fn open_image()

The central loading method. Checks the preload cache first for an instant hit. Otherwise sets loading_path, updates the navigator, and spawns a background thread. The thread converts OpenMode to a resize: bool for the channel message, calls load_image(), converts the result with .into(), posts the message, and fires WakeUp.

5.8 fn apply_image() → Option<(u32, u32)>

Stores the image, clears loading state, updates the navigator, and starts preloading neighbours. Returns Some((w, h)) if this is the first image and the window should resize, None otherwise. Returning resize intent as a value — rather than setting a field as a side effect — keeps the data flow explicit and testable.

5.9 fn preload_adjacent()

Called from inside apply_image() — they are logically one operation. Spawns up to two threads for the next and previous siblings. Each thread uses raw.into() to convert the decode result and sends a LoadMsg::Preloaded message.

5.10 fn navigate(forward: bool)

Replaces the former navigate_next and navigate_prev pair. The two methods were identical except for calling .next() vs .prev(). A single method with a direction argument is cleaner. true/false for forward/backward is universally understood and the call sites (app.navigate(true, &proxy)) read clearly.

5.11 fn delete_current()

Renamed from confirm_delete to better reflect its actual role — by the time this is called, confirmation has already happened via the native dialog. Deletes the file, removes it from the navigator, clears the preload cache (neighbours are now stale), then navigates to the next sibling or clears the view if none remain. Errors go directly to show_error_dialog().

5.12 fn process_messages() → Option<(u32, u32)>

Drains the crossbeam channel with non-blocking try_recv(). Three clean match arms, each doing exactly one thing. The return value propagates the resize intent upward to the event loop — the only place that can issue window commands — keeping the App struct free of window-management state.

5.13 fn render()

The pixel renderer. Writes directly into softbuffer's u32 slice. When no image is present, fills with pack_rgb(BG, BG, BG). When an image is present:

- scale = min(win_h / img_h, win_w / img_w) — fit without cropping.
- Destination rectangle centred in the window with equal margins.
- Nearest-neighbour sampling: sx = (dx - off_x) / scale. Clamped to image bounds on the same line as the index computation rather than as separate variables.
- Alpha compositing: out = (src * alpha + BG * (255 - alpha)) / 255. Uses the named BG constant rather than the magic literal 42.

5.14 fn confirm_delete_dialog() / fn show_error_dialog()

Both call MessageBoxW from the Windows API via **windows-sys**. The call is blocking — the entire event loop pauses while the dialog is open, which is correct for a modal confirmation. The UTF-16 conversion (encode_wide().chain(Some(0))) is required because the Windows W-suffix API uses UTF-16, not UTF-8. #[cfg(not(windows))] fallbacks keep the code compilable on other platforms.

5.15 fn monitor_usable() / fn clamp_to_monitor() / fn center_window()

monitor_usable() is a shared helper extracted because both clamp_to_monitor and center_window previously queried the monitor independently. Now the monitor is queried once and the result is shared. It returns 90% of width and 85% of height to leave room for the taskbar and title bar.

clamp_to_monitor() scales the image dimensions down to fit within the usable area if necessary.

center_window() sets the window's outer position to centre it on the monitor, with a minimum Y offset of 30 × scale_factor pixels to keep the title bar on screen.

5.16 The winit event loop in main()

ControlFlow::Wait means the loop sleeps until an event arrives — zero CPU when idle. The key events:

- **Event::UserEvent(WakeUp)** — background thread finished. Drain messages. If process_messages returns resize dimensions, issue request_inner_size and set pending_center. On the next WakeUp, call center_window. Then request a redraw.
- **WindowEvent::Resized** — request a redraw so the image rescales to the new window size.
- **WindowEvent::DroppedFile** — canonicalize and open in Navigate mode (no window resize).

- **WindowEvent::KeyboardInput** — matched on PhysicalKey::Code (hardware position, layout-independent). Navigate keys call app.navigate(true/false). Delete key calls confirm_delete_dialog() synchronously then app.delete_current() if confirmed.
- **WindowEvent::RedrawRequested** — resize the softbuffer surface, call render(), call present().

6. Threading & Asynchronous Design

6.1 Why threads at all?

Image decoding is both I/O-bound (disk reads) and CPU-bound (decompression, rasterisation). A large TIFF or a complex SVG can take hundreds of milliseconds to decode. Doing this on the event loop thread would freeze the window completely — no input, no redraw, no response. All decoding is moved to background threads.

6.2 The channel pipeline

- One bounded crossbeam channel, capacity 32, created in main().
- The Sender is cloned for each background thread. Cloning a Sender is cheap — it increments a reference count.
- The Receiver is drained in the event loop on each Event::UserEvent(WakeUp).
- Background threads post a LoadMsg then call proxy.send_event(WakeUp) to inject a synthetic event and wake the sleeping loop immediately.

■ *proxy.send_event() is thread-safe — the EventLoopProxy is an Arc internally. Cloning it for each thread is cheap and safe.*

6.3 Why ControlFlow::Wait matters

The previous egui version repainted continuously at up to 60 fps. winit with ControlFlow::Wait only wakes when an event arrives — OS input, a proxy.send_event() from a background thread, or a resize. When the user is not interacting, the process uses effectively zero CPU. This is the correct behaviour for a passive viewer.

6.4 Automatic Send — no unsafe needed

In Rust, a type is Send (safe to transfer to another thread) automatically if all its fields are Send. DecodedImage contains only Vec<u8> and u32, both of which are Send. LoadMsg contains only Send types. Therefore both are Send automatically — the compiler verifies this without any annotation.

An earlier version had unsafe impl Send for LoadMsg {}. This was incorrect: writing unsafe impl for a trait the compiler would have derived automatically bypasses the safety analysis without providing any benefit, and is considered a code smell. The correct approach — and the final one — is simply to let the compiler derive it.

6.5 The preload cache — CPU memory only

Decoded pixels live in Vec<u8> and are read directly by render(). There is no GPU upload step. Background threads produce the final display-ready data and the UI thread reads it directly. This is simpler than the egui architecture, which required keeping decoded pixels in CPU memory until the UI thread could upload them to the GPU.

6.6 Stale result discarding

If the user navigates faster than images decode, multiple Loaded messages may arrive. Each is checked against loading_path: if the path does not match what we are waiting for, the message is silently discarded. This prevents showing the wrong image when the user has skipped ahead.

6.7 The two-event resize/centre sequence

When window.request_inner_size() is called, the OS processes the resize asynchronously. If center_window() is called in the same event handler, it reads the window's pre-resize outer size and computes the wrong position — often placing the title bar off-screen.

The fix: on the WakeUp that delivers the first image, call request_inner_size() and set pending_center = true. On the *next* WakeUp, the resize has been applied and center_window() is accurate. window.request_redraw() ensures the next event arrives immediately rather than waiting for user input.

7. Key Design Decisions & Trade-offs

winit + softbuffer over egui/eframe

The primary architectural decision. egui is a full immediate-mode GUI toolkit — font atlas, widget layout engine, OpenGL backend. For a viewer that needs to display one image and respond to a handful of keys, this was significant over-engineering. Switching to winit + softbuffer reduced the dependency graph from ~200 crates to ~50–70 and the binary from ~9 MB to ~2–3 MB.

The trade-off: no built-in widgets. Text rendering, spinners, and any future UI elements must be implemented manually. For this application the only UI needed was a delete confirmation dialog, which is better served by a native OS call anyway.

Return values over side effects

Two methods return meaningful values instead of writing to fields as side effects: apply_image() returns Option<(u32, u32)> and process_messages() propagates it. The event loop — the only place that can issue window commands — receives the resize intent as a value rather than polling a field. This keeps the App struct free of window-management state and makes data flow explicit.

Types over booleans

OpenMode (FirstImage/Navigate) replaces a bool argument. Three distinct LoadMsg variants replace a Ready { is_preload: bool } field. The From trait replaces a bespoke constructor. In each case the refactoring makes the intent visible at the call site and removes conditional logic from match arms.

Native OS dialogs

The delete confirmation and error dialogs use MessageBoxW directly. This is a blocking call — exactly correct for modal confirmations. It requires zero rendering code, inherits the system theme, and is what Windows users expect. It also eliminates the show_delete_dialog flag that existed in earlier versions to suppress navigation 'while the dialog is open': since MessageBoxW blocks the event loop, navigation events physically cannot arrive.

Previous image held during navigation

current_image is not cleared when navigation starts. The user sees the previous image until the new one is ready. Combined with preloading, most navigations are instant. For slow storage or large files, the old image provides a stable visual anchor rather than an empty window.

navigate(forward: bool) over two methods

The former navigate_next and navigate_prev were identical one-liners differing only in .next() vs .prev(). A single method with a direction argument is more honest about what is actually happening — one operation with two modes — and reduces the surface area of the public API.

8. Caveats & Known Limitations

File deletion is synchronous

std::fs::remove_file() blocks the event loop. On a local SSD this is instant. On a network drive it could cause a brief freeze. Moving deletion to a background thread would fix this.

Render loop is O(win_w × win_h)

The render function iterates every pixel on every redraw. For a 4K window this is ~8 million iterations. Fast enough on any modern CPU, but could be parallelised with rayon or accelerated with SIMD for very large windows or images.

Nearest-neighbour scaling

Chosen for simplicity and speed. For photographic content at typical viewing scales the quality difference from bilinear is imperceptible. For extreme upscaling of small images (e.g. a 16×16 ICO in a 4K window) bilinear would look better. Could be added as an opt-in.

No zoom or pan

The viewer always fits the image to the window. Zoom/pan would require tracking offset and scale state and handling scroll wheel events.

Animated GIF plays only the first frame

The image crate decodes only frame 0. Full animation would require a timer, per-frame delay metadata, and scheduled redraws.

monitor_usable uses physical size not work area

winit provides the full monitor size, not the work area (screen minus taskbar). The 90%/85% multipliers are a heuristic. On unusual setups a Win32 SystemParametersInfo(SPI_GETWORKAREA) call would be more precise.

JXL HDR content is tone-mapped by clamping

f32_to_u8 clamps values above 1.0. A proper tone-mapping operator (Reinhard, ACES) would handle HDR JXL content more gracefully.

SVG external resources not loaded

usvg with default options does not resolve external resources (linked fonts, external images). SVGs referencing these may render incompletely.

9. Glossary of Rust Concepts Used

Ownership
Every value has exactly one owner. When the owner goes out of scope the value is dropped. No garbage collector needed.

Borrowing
You can lend a reference (&T;) without transferring ownership. Rust guarantees at compile time that references never outlive the value they point to.

&mut; self vs &self;
&mut; self = exclusive mutable access. &self; = shared read-only access. The compiler enforces that both cannot coexist simultaneously.

Option
Either Some(value) or None. Rust's safe replacement for null pointers. You cannot use the value without explicitly handling the None case.

Result
Either Ok(value) or Err(error). The ? operator propagates errors up the call stack automatically.

Vec
A growable heap-allocated array.

HashMap
A hash table. Lookup, insert, and delete are O(1) average.

Arc
Atomically Reference Counted smart pointer. Allows shared ownership across threads. Value is dropped when the last Arc is dropped.

Enum with data
Rust enums carry data per variant. LoadMsg::Loaded carries path, image, and a bool. LoadMsg::Error carries a string. Unlike C enums which carry no data.

From / Into traits
Standard conversion traits. impl From for DecodedImage lets you write raw.into() and Rust calls the implementation automatically.

Derive macros
#[derive(Clone, Debug)] auto-generates Clone and Debug implementations. The compiler writes the code; you just annotate the type.

Send
A marker trait indicating a type is safe to transfer to another thread. Vec, u32, PathBuf, String, and bool are all Send. Types containing only Send types are also Send automatically — no unsafe annotation needed.

unsafe
A keyword that allows a small set of operations the compiler cannot verify automatically — typically calling C functions. The unsafe block around MessageBoxW tells the compiler 'I guarantee these pointer arguments are valid'. It should be used only when necessary and kept as small as possible.

ControlFlow
An enum returned from the winit event loop closure. ControlFlow::Wait tells the loop to sleep until an event arrives. ControlFlow::Poll would spin at ~60fps continuously.

EventLoopProxy
A cloneable, thread-safe handle to the winit event loop. proxy.send_event() injects a synthetic event from any thread, waking the sleeping loop.

PhysicalKey::Code
winit represents keys by their hardware position (KeyCode), independent of keyboard layout. ArrowRight is always ArrowRight on QWERTY, AZERTY, Dvorak, etc.

NonZeroU32
A u32 guaranteed to be non-zero at compile time. softbuffer requires this for resize dimensions to prevent zero-size surface allocations.

type alias
type Proxy = winit::event_loop::EventLoopProxy; creates a shorter name for a verbose type. No runtime cost — purely a compile-time convenience.

cfg(windows)
Conditional compilation. Code inside #[cfg(windows)] only compiles on Windows. Used for MessageBoxW so the binary compiles on macOS and Linux too.

Newtype / zero-size struct
struct WakeUp; has no fields and no size at runtime. It exists purely as a type-level marker to parameterise EventLoop, giving the custom event a distinct type.